

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет
«Запорізька політехніка»

МЕТОДИЧНІ ВКАЗІВКИ
до виконання лабораторних робіт
з дисципліни
“Створення інтегрованих
середовищ розробки”
для студентів спеціальностей
121 "Інженерія програмного забезпечення" та
122 "Комп'ютерні науки"
(всіх форм навчання)

2023

Методичні вказівки до виконання лабораторних робіт з дисципліни "Створення інтегрованих середовищ розробки" для студентів спеціальностей 121 "Інженерія програмного забезпечення" та 122 "Комп'ютерні науки" (всіх форм навчання) / В.М. Льовкін. – Запоріжжя: НУ "Запорізька політехніка", 2023. – 42 с.

Автор: В.М. Льовкін, канд. техн. наук, доцент

Рецензент: А.О. Олійник, д-р. техн. наук, професор

Відповідальний
за випуск: С.О. Субботін, д-р. техн. наук, професор

Затверджено
на засіданні кафедри
програмних засобів

Протокол № 8
від “30” березня 2023 р.

ЗМІСТ

Вступ	5
Лабораторна робота № 1	6
Лексичний аналізатор	6
1.1. Мета роботи	6
1.2. Короткі теоретичні відомості	6
1.2.1 Лексичний аналіз	6
1.2.2 Генератор парсерів ANTLR. Побудова лексичних аналізаторів	7
1.3. Завдання на лабораторну роботу.....	20
1.4. Зміст звіту.....	20
1.5. Контрольні запитання	21
Лабораторна робота № 2	23
Синтаксичний аналізатор.....	23
2.1. Мета роботи	23
2.2. Короткі теоретичні відомості	23
2.2.1 Синтаксичний аналіз	23
2.2.2 Генератор парсерів ANTLR. Синтаксичний аналізатор.....	24
2.3. Завдання на лабораторну роботу.....	28
2.4. Зміст звіту.....	29
2.5. Контрольні запитання	29
Лабораторна робота № 3	31
Семантичний аналізатор.....	31
3.1. Мета роботи	31
3.2. Короткі теоретичні відомості	31
3.3. Завдання на лабораторну роботу.....	31
3.4. Зміст звіту.....	33
3.5. Контрольні запитання	33
Лабораторна робота № 4	35
Генератор коду	35
4.1. Мета роботи	35
4.2. Короткі теоретичні відомості	35
4.3. Завдання на лабораторну роботу.....	36
4.4. Зміст звіту.....	36
4.5. Контрольні запитання	37
Додаток А.....	38

Індивідуальні завдання	38
Література.....	42

ВСТУП

Дане видання призначене для вивчення та практичного освоєння студентами основ створення інтегрованих середовищ розробки, що включає в себе питання лексичного, синтаксичного і семантичного аналізу, оптимізації і генерації коду та побудови компіляторів загалом.

Відповідно до графіка студенти перед виконанням лабораторної роботи повинні ознайомитися з конспектом лекцій та рекомендованою літературою. Дані методичні вказівки містять тільки основні, базові теоретичні відомості, необхідні для виконання лабораторних робіт, тому для виконання лабораторної роботи та при підготовці до її захисту необхідно ознайомитись з конспектом лекцій та опрацювати необхідний матеріал, наведений в переліку рекомендованої літератури, використовуючи, коли це необхідно, також статті в актуальних наукових журналах з інженерії програмного забезпечення.

Для одержання оцінки з кожної роботи студент повинен у відповідності зі всіма наведеними вимогами розробити програмне забезпечення та оформити звіт, після чого продемонструвати на комп'ютері розроблене програмне забезпечення з виконанням всіх запропонованих викладачем тестів або надати його для автоматизованого тестування, захистити виконану роботу. За друкованого варіанту представлення віт виконують на білому папері формату А4. Текст розміщують тільки з однієї сторони аркуша. Поля сторінки з усіх боків – 20 мм. Аркуші вміщують у канцелярський файл.

Усі завдання повинні виконуватись студентами індивідуально і не містити ознак плагіату як в оформленому звіті, так і в розробленому програмному забезпеченні. Під час співбесіди при захисті лабораторної роботи студент повинен виявити знання щодо мети роботи, теоретичного матеріалу, методів виконання кожного етапу роботи, змісту основних розділів звіту з демонстрацією результатів на конкретних прикладах, практичних прийомів використання теоретичного матеріалу в розробленому програмному забезпеченні. Студент повинен вміти обґрунтувати всі прийняті ним рішення та правильно аналізувати і використовувати на практиці отримані результати. Для базової самоперевірки при підготовці до виконання і захисту роботи студент повинен відповісти на контрольні запитання, наведені наприкінці опису відповідної роботи.

ЛАБОРАТОРНА РОБОТА № 1 ЛЕКСИЧНИЙ АНАЛІЗАТОР

1.1. Мета роботи

1.1.1 Ознайомитись з основами процесу виконання лексичного аналізу вихідної програми та інструментами, які використовуються під час реалізації даного процесу.

1.1.2 Навчитися проєктувати та розробляти лексичний і синтаксичний аналізатори у складі компілятора за допомогою генератора парсерів ANTLR.

1.2. Короткі теоретичні відомості

1.2.1 Лексичний аналіз

Процес компіляції програми розпочинається з лексичного аналізу є першим етапом процесу компіляції. На даному етапі зчитується вхідний текст програми, представлений послідовністю символів, в результаті формуючи послідовність термінальних символів мови, яку використовує синтаксичний аналізатор, що реалізується на другому етапі процесу компіляції.

На рис. 1.1 схематично зображено зв'язок першого та другого етапів процесу компіляції, тобто лексичного та синтаксичного аналізу, та те, які дії забезпечують аналіз вхідного тексту програми, написаного відповідною мовою програмування.



Рисунок 1.1 – Зв'язок лексичного та синтаксичного аналізу

Лексичний аналізатор для своєї роботи потребує визначення лексичної граматики, на основі якої як раз і відбувається виділення

токенів. Лексична граматики має бути задана в підсумку за допомогою регулярних виразів.

Регулярні вирази мають бути перетворені на недерміновані кінцеві автомати (НКА), а НКА далі в свою чергу мають бути перетворені на детерміновані кінцеві автомати (ДКА).

Під час роботи лексичного аналізатора також забезпечується таблиця символів. Результати виділення ідентифікаторів з коду програми призводять до збереження даних про них у таблиці символів, а потім ця таблиця використовується далі для аналізу.

1.2.2 Генератор парсерів ANTLR. Побудова лексичних аналізаторів

Генератор парсерів ANTLR (ANother Tool for Language Recognition) [8-9] дозволяє автоматизовано генерувати лексичний аналізатор, написаний мовою програмування Java (або іншою доступною мовою) на основі граматики вхідної мови програмування або іншої мови, при цьому таку граматику необхідно початково визначити.

Існує декілька способів, якими можна використовувати можливості ANTLR.

Перший спосіб полягає у використанні засобів ANTLR без застосування середовища розробки. Для того щоб інсталивати ANTLR в операційних системах сімейства Windows, необхідно виконати наступні дії:

- а) встановити засоби Java-середовища (JDK);
- б) завантажити архів з ANTLR відповідної версії (наприклад, <https://wwwantlr.org/download/antlr-4.13.1-complete.jar>), зберігши його у окремо створену теку в корені диску (наприклад, C:\Javalib);
- в) створити змінну середовища для користувача CLASSPATH, в яку внести відповідне значення зі шляхом до завантаженого JAR-файлу (C:\Javalib\antlr-4.13-complete.jar);
- г) додати до змінної середовища для користувача Path створену теку (C:\Javalib);
- ґ) додати до системної змінної Path шлях до JDK (наприклад, C:\Program Files\Java\jdk-17.0.1\bin);
- д) створити короткі команди для швидкого запуску для ANTLR

Tool, використовуючи командні файли:

– antlr4.bat:

CMD

```
java org.antlr.v4.Tool %*
```

– grun.bat:

CMD

```
java org.antlr.v4.runtime.misc.TestRig %*
```

Для тестування результатів налаштування необхідно запустити org.antlr.v4.Tool напряму

CMD

```
$ java org.antlr.v4.Tool
```

або використовуючи –jar опцію java:

CMD

```
$ java -jar C:\Javalib\antlr-4.13.1-complete.jar
```

Отриманий при цьому результат показаний на рис. 1.2.

У подальшому потрібно створити у теці файл граматики з розширенням g4 (наприклад, Example.g4). Тоді необхідно згенерувати java-код за допомогою команди:

CMD**\$ antlr Example.g4**

```

c:\Javalib>java -jar C:\Javalib\antlr-4.13.1-complete.jar
ANTLR Parser Generator Version 4.13.1
  -o _____ specify output directory where all output is generated
  -lib _____ specify location of grammars, tokens files
  -atn _____ generate rule augmented transition network diagrams
  -encoding _____ specify grammar file encoding; e.g., euc-jp
  -message-format _____ specify output style for messages in antlr, gnu, vs2005
  -long-messages _____ show exception details when available for errors and warnings
  -listener _____ generate parse tree listener (default)
  -no-listener _____ don't generate parse tree listener
  -visitor _____ generate parse tree visitor
  -no-visitor _____ don't generate parse tree visitor (default)
  -package _____ specify a package/namespace for the generated code
  -depend _____ generate file dependencies
  -Doption=value _____ set/override a grammar-level option
  -Werror _____ treat warnings as errors
  -XdtgST _____ launch StringTemplate visualizer on generated code
  -XdtgSTwait _____ wait for STViz to close before continuing
  -Xforce-atn _____ use the ATN simulator for all predictions
  -Xlog _____ dump lots of logging info to antlr-timestamp.log
  -Xexact-output-dir _____ all output goes into -o dir regardless of paths/package

c:\Javalib>_

```

Рисунок 1.2 – Перевірка налаштування ANTLR

Після цього потрібно згенерувати class-файли за допомогою команди:

CMD**\$ javac Example*.java**

З даного коду можна згенерувати дерево, але детальніше це буде розглянуто в наступній роботі (для перевірки можна створити файл `example.txt` з прикладом вхідного коду, який має бути оброблений, а тоді потрібно виконати команду `grun Example r example.txt -gui` для побудови у графічному вигляді дерева, де `r` – назва кореневого елемента).

Сам же створений програмний код можна використовувати у власних проектах, використовуючи відповідні класи та організуючи звернення до них для аналізу коду. Тоді середовище буде

використовуватися вже безпосередньо для роботи з Java-проектом.

Окрім того ANTLR доступний зокрема у якості плагінів для IntelliJ, NetBeans і Eclipse.

У середовищі IntelliJ IDEA необхідно створити звичайний Java-проект, встановити відповідний плагін. Для встановлення плагіна необхідно з меню File вибрати підменю Settings, звідки обрати пункт Plugins, а в пошуку задати antlr і встановити відповідну версію плагіна (рис. 1.3).

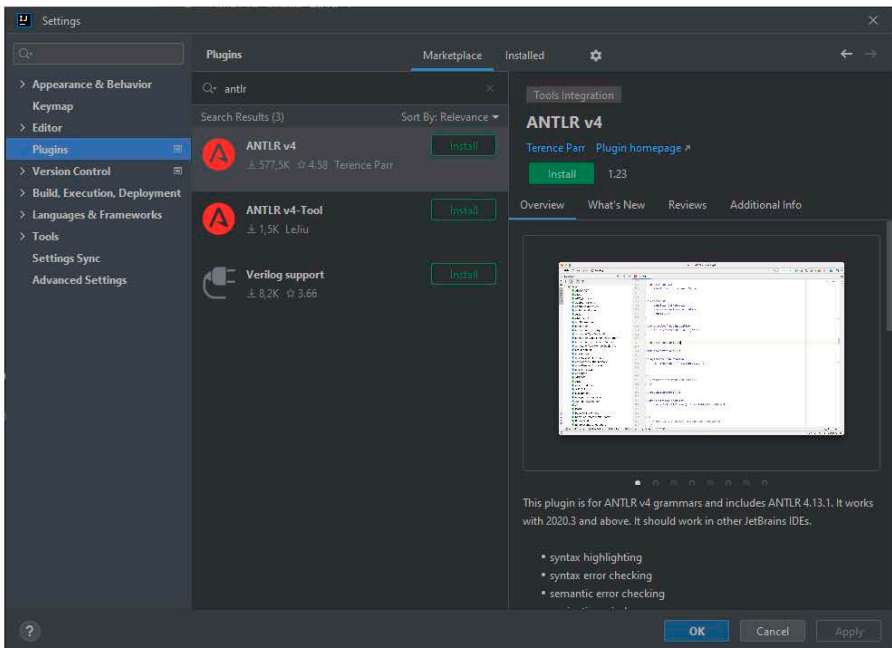


Рисунок 1.3 – Встановлення плагіна ANTLR у середовищі IntelliJ IDEA

Після встановлення плагіна необхідно в середовищі до проекту додати файл з граматикою з розширенням g4, який створюється як звичайний файл. Після того, як граматика повністю визначена, необхідно обрати з контекстного меню пункт Generate ANTLR Recognizer (рис. 1.4), що дозволить створити код самого аналізатора (або відповідно парсера).

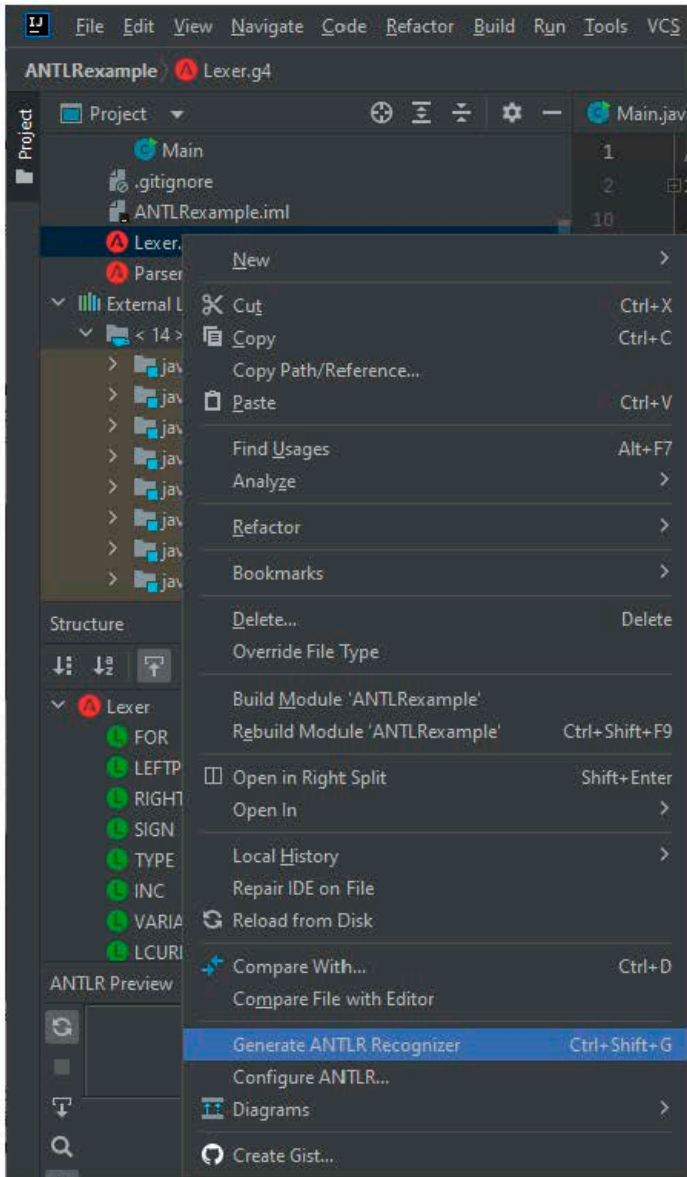


Рисунок 1.4 – Генерування коду в середовищі IntelliJ IDEA

Сам згенерований код знаходиться в теці gen.

У випадку підключення

Для того щоб підключити плагін ANTLR у середовище Eclipse, необхідно підключити плагіни Xtext (рис. 1.5) і ANTLR 4 IDE.



Рисунок 1.5 – Встановлення плагіна Xtext

- Для встановлення плагіна необхідно виконати наступні дії:
- обрати у Eclipse пункт меню Help → Eclipse Marketplace...;
 - задати в полі для пошуку Xtext або antlr у другому випадку (рис. 1.6);
 - обрати у результатах пошуку Xtext для першого випадку або ANTLR 4 IDE для другого випадку та натиснути на Install.

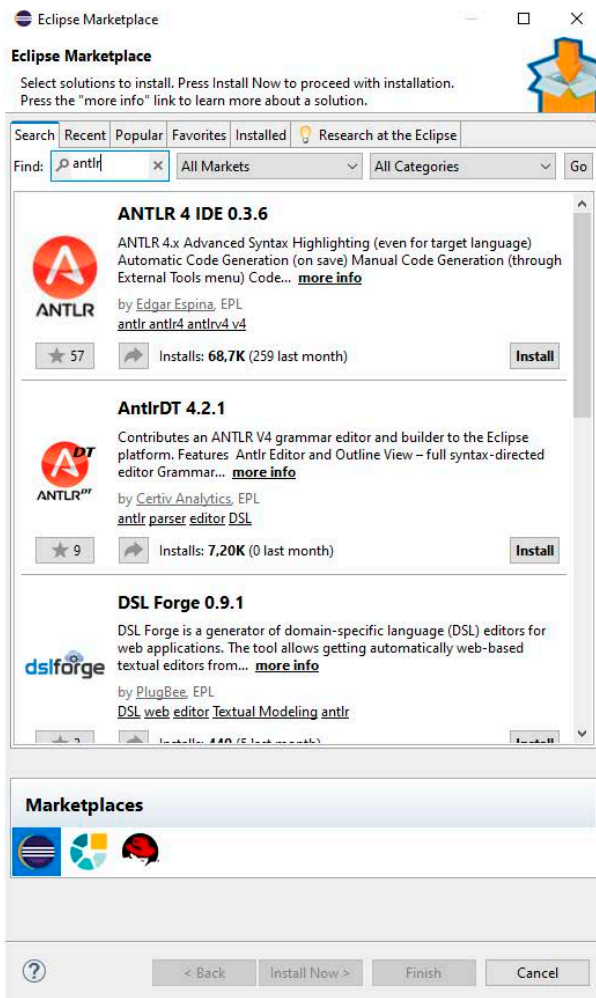


Рисунок 1.6 – Результати пошуку плагіну ANTLR у Eclipse

Після того, як встановлення завершено, потрібно створити новий проєкт, обираючи з меню File пункт New, а далі Project..., звідки з групи General потрібно обрати тип проєкту ANTLR 4 Project (рис. 1.7).

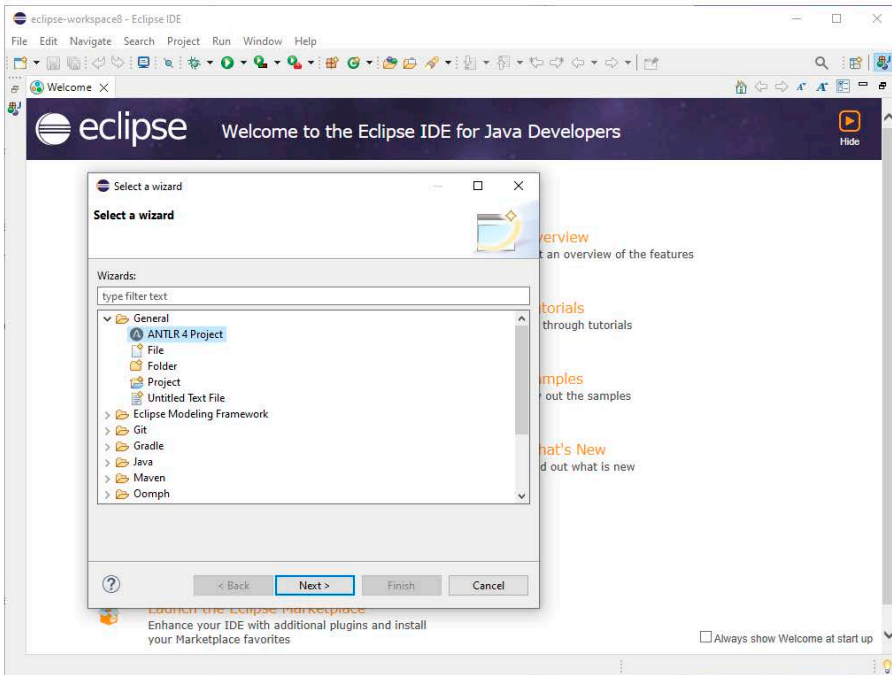


Рисунок 1.7 – Створення нового проєкту ANTLR 4

У файлах проєкту ANTLR 4 з розширенням *.g4 можна визначити необхідну граматику мови.

Після того, як створено необхідну граматику, потрібно зберегти останню версію проєкту, а далі за необхідності з контекстного меню граматики обрати пункт Run As та підпункт Generate ANTLR Reecognizer (рис. 1.8). За замовчанням код генерується автоматично під час збереження, якщо граматика вірно задана.

Для налаштування генератора необхідно обрати Window → Preferences → ANTLR 4 → Tool.

Після цього буде створено файли проєкту на Java, у яких реалізовано лексичний аналізатор для заданої граматики. Файли містяться у теці `target/generated-sources/antlr4` проєкту.

При підготовці наступної частини використано документацію [9].

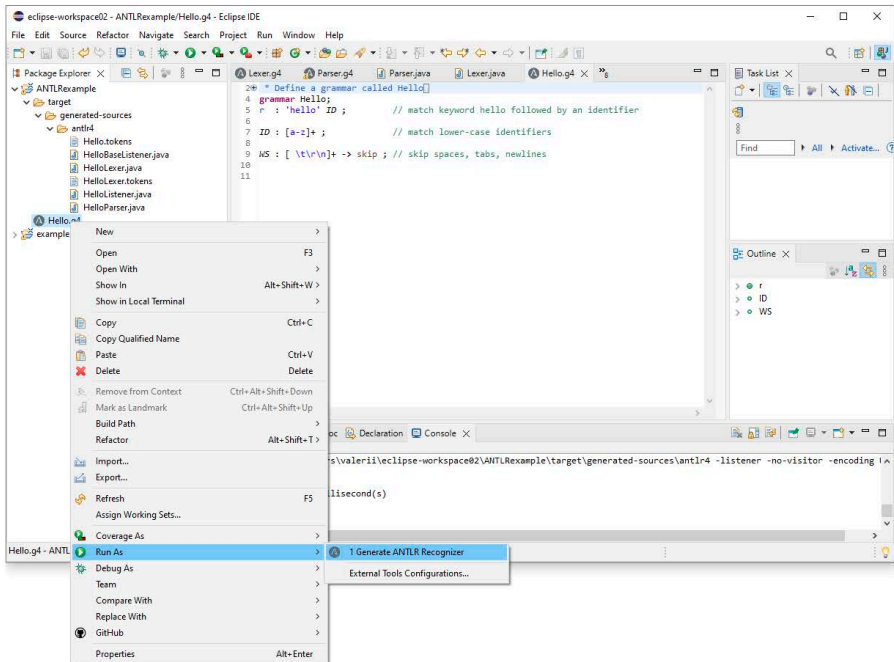


Рисунок 1.8 – Генерація коду за проєктом ANTLR 4 у Eclipse

Загальна форма представлення граматики у ANTLR 4 наступна [9]:

ANTLR

```

grammar Name;
options {...}
import ... ;
tokens {...}
@actionName {...}

```

ANTLR

```
rule1//правила лексичного та синтаксичного аналізатора
...
ruleN
```

Заголовок та щонайменше одне правило мають бути обов'язково визначені у файлі граматики. Правила представляються у наступній формі:

ANTLR

```
LexerRuleName : alternative1 | ... | alternativeN ;
```

Назви правил синтаксичного аналізатора мають починатися з маленької літери, а лексичного – з великої.

Для того щоб визначити граматичку, яка містить тільки правила для лексичного розбору, необхідно використовувати наступний заголовок:

ANTLR

```
lexer grammar Name;
```

Розділ `import` дозволяє розділяти визначення граматички на частини та оброблює у об'єктно-орієнтованому стилі, наслідуючи всі правила, визначення токенів та дії з імпортованих граматик.

У розділі токенів визначаються типи токенів, які розглядаються в даній граматичці та для яких немає асоційованих правил лексичного розбору. Синтаксис визначення наступний:

ANTLR

```
tokens { Token1, ..., TokenN }
```


Лексичні правила формують визначення токенів та більш-менш відповідають синтаксису синтаксичних правил за винятком того, що дані правила не можуть мати аргументів, локальних змінних або повертати значення.

Можна також визначати правила, які не є токенами, але дозволяють визначати токени, наприклад [9]:

ANTLR

```
INT : DIGIT+ ;
fragment DIGIT : [0-9] ;
```

Режими дозволяють групувати лексичні правила за контекстом:

ANTLR

```
rules in default mode
...
mode MODE1;
rules in MODE1
...
mode MODEN;
rules in MODEN
...
```

Елементами лексичних правил можуть бути:

- 'literal' – символ або набір символів;
- [char set] – один з символів, зазначених у множині (множина x-у інтерпретується як інтервал від x до y);
- 'x'..'y' – будь-який символ у діапазоні між x та y;
- T – викликати правило T (дозволено рекурсію, але не лівобічну);
- {«action»} – семантична дія;
- {«p»}? – семантичний предикат;
- ~x – будь-який символ, що не входить у вказану множину x;

– . – будь-який символ.

Лексичні правила можуть бути в тому числі й рекурсивними за природою [9]:

ANTLR

*ACTION : '{ (ACTION | ~[}]) * }' ;*

Для того щоб уникнути прив'язування граматики до конкретної цільової мови, ANTLR підтримує команди лексичного аналізатора: може бути тільки одна така команда у одному правилі токена, синтаксис визначення таких правил наступний [9]:

ANTLR

TokenName : «alternative» -> command-name («identifier or integer»)

Вірні імена команд:

- skip – перейти до наступного токена, відкидаючи поточний текст;
- more – перейти до наступного токена, не відкидаючи поточний текст;
- mode(x), pushMode(x), popMode – змінюють стек режиму та сам режим;
- type(x) та channel(x) [9].

Розділ options дозволяє визначити опції наступним чином:

ANTLR

options { name1=value1; ... nameN=valueN; }

Дії – блоки тексту, написані цільовою мовою та обмежені фігурними дужками. Наприклад:

ANTLR

```
decl: datatype ID ';' {System.out.println("Here it is.");} ;
datatype: 'float' | 'double' ;
```

Всі токени мають набір попередньо визначених, доступних тільки для читання атрибутів:

– text – текст, що відповідає токену (транслюється у виклик getText);

– type – тип токену (додатне ціле число);

– line – номер рядка, на якому з'явився токен;

– pos – позиція першого символу токену в рядку;

– index – індекс токену;

– channel – номер каналу для токену;

– int – цілочислове значення тексту [9].

Приклад спрощеної граматики для проведення лексичного розбору:

ANTLR

```
lexer grammar Lexerexample;
```

```
FOR: 'for' ;
```

```
SIGN: '>'|'<'|'='|'>='|'<=' ;
```

```
TYPE: 'int' | 'short' | 'long' ;
```

```
INC: '++' ;
```

```
VARIABLE: [A-Za-z_]+ ;
```

```
SEMI: ';' ;
```

```
ASSIGN: '=' ;
```

```
NEWLINE: [\r\n]+ -> skip;
```

```
NUMBER: [0-9]+ ;
```

1.3. Завдання на лабораторну роботу

1.3.1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.

1.3.2. Визначити граматику для виконання лексичного аналізу програм мовою, заданою у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем.

1.3.3. Побудувати кінцеві автомати для представлення сформованої граматики.

1.3.4. Побудувати лексичний аналізатор за допомогою генератора ANTLR мовою програмування Java:

- програма повинна виконувати лексичний аналіз вхідного тексту програми у відповідності з визначеним індивідуальним завданням;
- програма повинна обробляти і виводити повідомлення про всі помилки, виявлені під час лексичного аналізу з зазначенням причини помилки (недопустимі символи, пропущені лапки тощо);
- програма повинна мати графічний інтерфейс та дозволяти задавати вхідний код програми для аналізу, задаючи його безпосередньо в інтерфейсі або створюючи файл програми, завантажуючи його через інтерфейс та виводячи результати лексичного аналізу.

1.3.5. Створити git-репозиторій для роботи над проектом та додати в нього розроблений лексичний аналізатор.

1.3.6. Виконати тестування побудованого лексичного аналізатора.

1.3.7. Оформити звіт.

1.3.8. Відповісти на контрольні запитання.

1.4. Зміст звіту

1.4.1. Мета роботи.

1.4.2. Завдання до роботи.

1.4.3. Лексична граматика.

1.4.4. Основні кінцеві автомати, які використовуються під час лексичного аналізу.

- 1.4.5. Текст лексичного аналізатора з основними коментарями.
- 1.4.6. Інтерфейс роботи з програмою в декількох режимах.
- 1.4.7. Результати тестування: вхідні дані та результати роботи програми.
- 1.4.8. Перелік проблем, які були виявлені і розв'язані або не розв'язані під час роботи над проектом, з описом відповідної ситуації. Якщо проблему було розв'язано, то необхідно додатково описати прийняті рішення. Якщо проблему не було розв'язано, ситуацію має бути ідентифіковано максимально детально з поясненням виявлених причин.
- 1.4.9. Висновки, що відображають особисто отримані результати виконання роботи та їх критичний аналіз.

1.5. Контрольні запитання

- 1.5.1. З чого складається сучасне інтегроване середовище розробки?
- 1.5.2. Чим відрізняються компілятори та інтерпретатори?
- 1.5.3. Що є результатом та входом лексичного аналізатору?
- 1.5.4. Які помилки може визначити лексичний аналізатор?
- 1.5.5. Яким чином виконується лексичний аналіз?
- 1.5.6. Що таке токен, лексема?
- 1.5.7. Як виділити токени мови програмування?
- 1.5.8. Що таке контекстно-вільна граматики?
- 1.5.9. Які бувають типи формальних мов?
- 1.5.10. Які операції можна виконати над регулярними мовами?
- 1.5.11. Які оператори можуть використовуватися для побудови регулярних виразів?
- 1.5.12. Що таке діаграми переходів?
- 1.5.13. За допомогою чого визначається кінцевий автомат?
- 1.5.14. Що таке НКА і ДКА та чим вони відрізняються?
- 1.5.15. Яким чином перетворити НКА на ДКА?
- 1.5.16. Які існують особливості виконання лексичного аналізу в залежності від мови програмування, трансляція якої відбувається?
- 1.5.17. Яка структура граматики у ANTLR?
- 1.5.18. Яким чином та які граматики можуть наслідуватися під час визначення граматики у ANTLR?

1.5.19. Яким чином створити лексичний аналізатор без використання графічних засобів ANTLR?

1.5.20. Які генератори лексичних аналізаторів відомі?

ЛАБОРАТОРНА РОБОТА № 2 СИНТАКСИЧНИЙ АНАЛІЗАТОР

2.1. Мета роботи

2.1.1 Ознайомитись з основними поняттями, методами та засобами синтаксичного аналізу вихідної програми.

2.1.2 Навчитися проектувати та розробляти синтаксичний аналізатор у складі компілятора за допомогою генератора парсерів ANTLR.

2.2. Короткі теоретичні відомості

2.2.1 Синтаксичний аналіз

Синтаксичний аналіз (або парсинг) є другим етапом процесу компіляції, що виконує співставлення лінійної послідовності лексем мови з її формальною граматикою. У загальному випадку результатом другого етапу процесу компіляції є дерево розбору, приклад якого наведено на рис. 2.1.

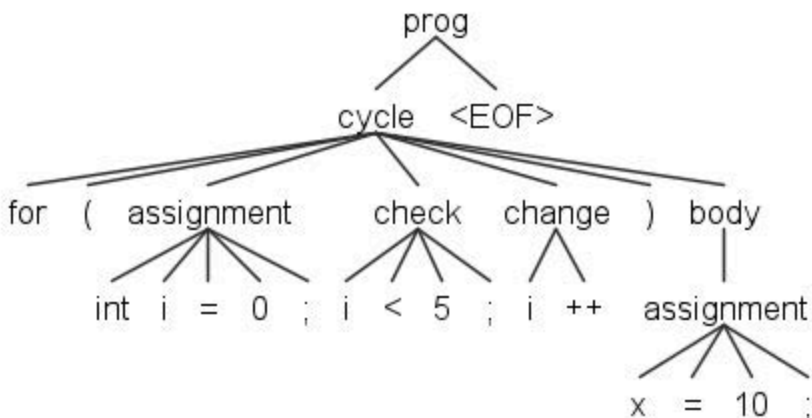


Рисунок 2.1 – Приклад дерева розбору, побудованого ANTLR

2.2.2 Генератор парсерів ANTLR. Синтаксичний аналізатор

Для розробленого аналізатора можна побудувати синтаксичне дерево або дерево розбору, задаючи при цьому текст програми.

За замовчуванням парсери, побудовані за допомогою ANTLR, будують структури даних, які представлені деревами розбору, що записують, яким чином парсер розпізнав структуру вхідного коду.

У пункті 1.2.2 було розглянуто правила побудови лексичної граматики та граматик загалом у генераторі парсерів ANTLR. Для того щоб визначити граматику, яка містить тільки правила для синтаксичного розбору, необхідно використовувати наступний заголовок:

ANTLR

```
parser grammar Name;
```

Стандартний синтаксис визначення правила синтаксичного аналізатора:

ANTLR

```
retstat : 'return' expr ';' ;
```

Альтернативи у правилах відділяються оператором `|` та можуть бути порожніми (пропускатися):

ANTLR

```
stat: retstat  
    | 'break' ';' ;  
    | 'continue' ';' ;  
    ;
```


Альтернативам також можна встановлювати мітки за допомогою оператора #.

Елементи правил:

- T – токен (починається з великої літери);
- 'literal' – рядковий літерал;
- r – правило;
- r [«args»] – правило з параметрами (перераховуються через кому);
- {«action»} – семантична дія;
- {«p»}? – семантичний предикат;
- . – будь-який токен окрім кінця файлу [9].

Правило може містити альтернативні блоки підправил у відповідності з розширеною формою Бекуса-Наура.

Для побудови парсера створюється відповідно друга частина граматики, яка безпосередньо ґрунтується на підготовленій першій частині граматики. Ці частини граматики виділяються в окремі g4-файли.

Для того щоб створити графічне представлення результатів розбору у вигляді дерева (рис. 2.1) у випадку роботи з операційної системи після виконання дій, необхідних для створення java і class-файлів, які були описані в першому розділі, необхідно виконати наступну команду:

CMD

```
$ grun Example r example.txt -gui
```

У випадку роботи з середовища у кожного з них є власні відповідні засоби.

Наприклад, в IntelliJ IDEA можна переглянути як структуру граматики, так і створене дерево (рис. 2.2), для чого призначена вкладка ANTLR Preview. На цій вкладці можна ввести необхідний для розбору текст, після чого створюється дерево, яке демонструє результати розбору.

Результати розбору можуть бути представлені візуально деревом, ієрархією з можливістю відкриття необхідних вузлів, а також

профайлером, який дозволяє переглянути параметри виконаного розбору.

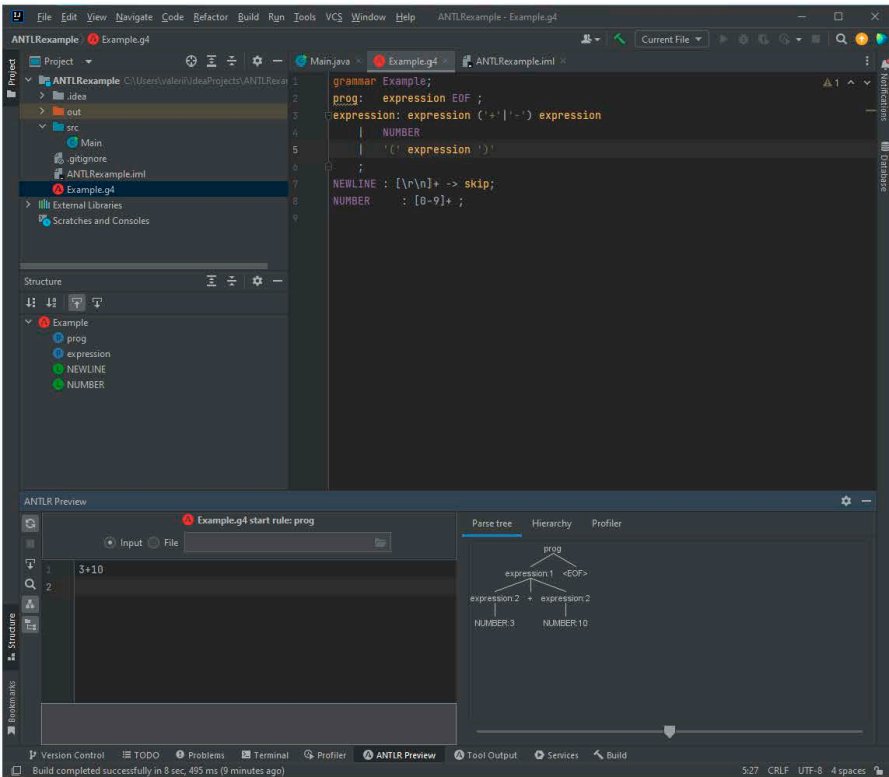


Рисунок 2.2 – Побудова дерева розбору в IntelliJ IDEA

Побудова дерева відбувається і в тому випадку, якщо виявлені помилки (рис. 2.3). Тоді відповідні позиції, які не вдалося розпізнати, показуються червоним. Це дозволяє додатково зрозуміти наявні проблеми та невідповідність тим граматикам, які були створені у проєкті.

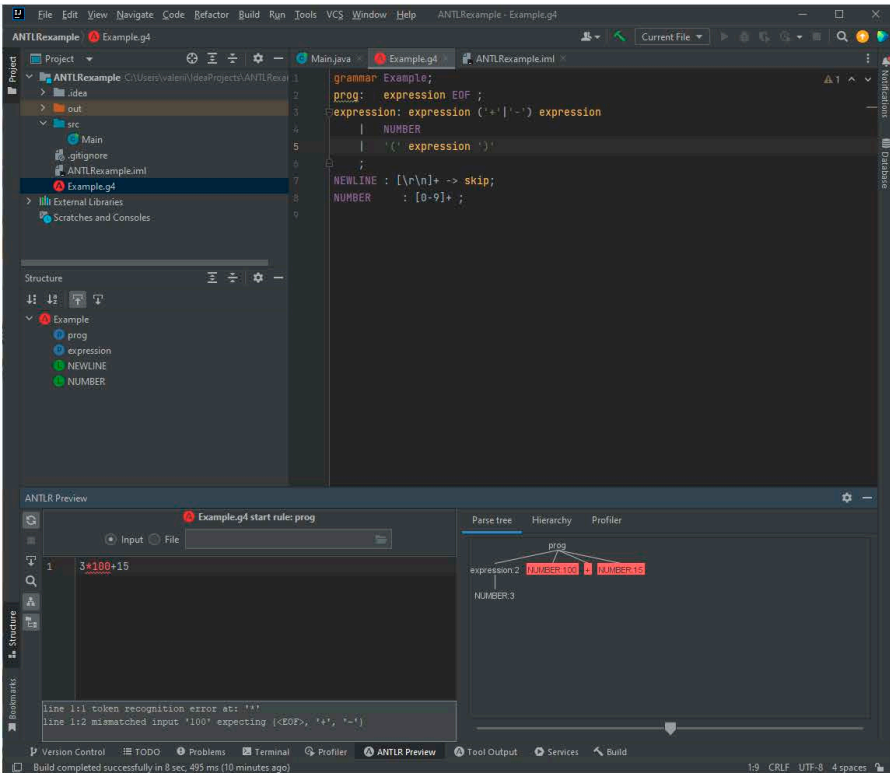


Рисунок 2.3 – Побудова дерева розбору в IntelliJ IDEA з виявленням помилок

У випадку появи синтаксичної помилки ANTLR кожне правило огорнуте конструкцією try/catch/finally:

Java

```

public final RContext r() throws RecognitionException {
    RContext _localctx = new RContext(_ctx,
    getState());
    enterRule(_localctx, 0, RULE_r);
    try {
        enterOuterAlt(_localctx, 1);
    }
  }
  
```

-- Java

```

        {
            setState(2); match(T__0);
            setState(3); match(ID);
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

```

2.3. Завдання на лабораторну роботу

2.3.1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.

2.3.2. Визначити граматику для виконання синтаксичного аналізу програм мовою, заданою у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем.

2.3.3. Розробити програмне забезпечення за допомогою генератора ANTLR мовою програмування Java на основі розробленого лексичного аналізатора, яке відповідає наступним вимогам:

- виконує синтаксичний аналіз тексту програми у відповідності з індивідуальним завданням;
- формує в результаті аналізу абстрактні синтаксичні дерева;
- виводить повідомлення про помилки, виявлені під час синтаксичного аналізу;
- має графічний інтерфейс та дозволяє виводити результати синтаксичного аналізу (передбачити також можливість роботи без графічного інтерфейсу).

2.3.4. Виконати тестування побудованого синтаксичного аналізатора.

2.3.5. Оформити звіт.

2.3.6. Відповісти на контрольні запитання.

2.4. Зміст звіту

2.4.1. Мета роботи.

2.4.2. Завдання до роботи.

2.4.3. Синтаксична граматики.

2.4.4. Текст синтаксичного аналізатора, згенерованого за допомогою генератора ANTLR, з основними коментарями.

2.4.5. Інтерфейс роботи з програмою в декількох режимах.

2.4.6. Результати тестування: вхідні дані та результати роботи програми.

2.4.7. Перелік проблем, які були виявлені і розв'язані або не розв'язані під час роботи над проектом, з описом відповідної ситуації. Якщо проблему було розв'язано, то необхідно додатково описати прийняті рішення. Якщо проблему не було розв'язано, ситуацію має бути ідентифіковано максимально детально з поясненням причин.

2.4.8. Висновки, що відображають особисто отримані результати виконання роботи, їх критичний аналіз та порівняння аналізаторів, побудованих вручну і за використання генератора.

2.5. Контрольні запитання

2.5.1. Які завдання розв'язуються в процесі синтаксичного аналізу?

2.5.2. Що таке контекстно-вільна граматики?

2.5.3. Чим відрізняються та в чому полягають низхідний та висхідний синтаксичний аналіз?

2.5.4. У чому полягає метод рекурсивного спуску?

2.5.5. У чому полягає нерекурсивний предикативний синтаксичний аналіз?

2.5.6. У чому полягає алгоритм LR-аналізу?

2.5.7. Що таке перенесення та згортка?

2.5.8. Що таке абстрактне синтаксичне дерево?

2.5.9. Які на даний момент існують генератори синтаксичних аналізаторів та чим вони відрізняються?

2.5.10. Яким чином побудувати синтаксичний аналізатор у ANTLR?

ЛАБОРАТОРНА РОБОТА № 3

СЕМАНТИЧНИЙ АНАЛІЗАТОР

3.1. Мета роботи

3.1.1 Ознайомитись з основними способами, засобами та методами, які використовуються під час семантичного аналізу вихідної програми на основі абстрактних синтаксичних дерев.

3.1.2 Навчитися проектувати та розробляти семантичний аналізатор у складі компілятора.

3.2. Короткі теоретичні відомості

Після проведення попередньої обробки тексту програми, виконаного через інтеграцію згенерованого коду лексичного і синтаксичного аналізаторів, далі над отриманими результатами реалізується семантичний аналіз. Семантичний аналіз виконується шляхом роботи з деревом розбору і оброблення цього дерева. У результаті виконання етапу семантичного аналізу формується також дерево, але це дерево проміжного представлення.

У процесі виявлення семантики коду завданням компілятора є також визначення всіх статичних помилок, включаючи також і семантичні та лексичні помилки. У підсумку має бути повідомлено про всі помилки, які таким чином були знайдені. Для виявлення таких помилок необхідне розуміння семантики коду, а не тільки синтаксису, як це було на попередньому етапі.

3.3. Завдання на лабораторну роботу

3.3.1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.

3.3.2. Програмно реалізувати формування таблиці символів.

3.3.3. Програмно реалізувати створення дерев проміжного представлення (вручну, додаючи відповідну функціональність у програму, або використовуючи в генераторі парсерів ANTLR при побудові граматики відповідні дії).

3.3.4. Реалізувати виконання програмним забезпеченням перевірки зокрема наступних правил (у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем) та генерацію відповідних інформативних повідомлень (файл програми, номер рядка, позиція в рядку, текст повідомлення) у разі виявлення помилок:

- жодний ідентифікатор (змінна, атрибут, функція або метод) не оголошено двічі в одній області видимості;
- жодний ідентифікатор (змінна, атрибут, функція або метод) не використано раніше ніж його оголошено в даній області видимості або без оголошення у ній;
- кількість та тип фактичних аргументів у виклику функції (методу) має бути такою ж як формальних параметрів у визначенні функції;
- при поверненні результату функцією значення має бути того ж типу, що і в оголошенні функції;
- методи та функції, які включені у вирази, мають повертати значення;
- керуючі оператори (break, continue) мають входити у тіло циклу;
- вираз, включений в умовний оператор, має бути логічного типу або перетворюватися у нього (у відповідності з правилами мови програмування);
- операнди операцій мають відповідати типам, для яких визначені дані операції;
- індекс масиву має бути цілочисельним;
- функції без типу не мають повертати значення;
- ієрархія класів (за умов їх присутності в граматиці мови) вірно побудована;
- зарезервовані ключові слова використано вірно.

Семантичні перевірки повинні виконуватися згори вниз. Тільки перевірка типів у складі семантичних перевірок може виконуватись у протилежному напрямку.

Усі виявлені помилки мають виводитись за один запуск.

3.3.5. Розширити семантичний аналіз виведенням попереджень у разі виявлення наступних умов:

- оголошена змінна не використовується жодного разу в програмі;
- оператори циклу не передбачають завершення циклу;
- умовні оператори не передбачають можливості руху всіма наявними своїми гілками.

Усі виявлені попередження мають виводитись за один запуск.

3.3.6. Відповідним чином модифікувати інтерфейс програмного забезпечення для виведення результатів виконання семантичного аналізу (п. 3.3.2–3.3.5).

3.3.7. Виконати тестування розробленого програмного забезпечення.

3.3.8. Оформити звіт.

3.3.9. Відповісти на контрольні запитання.

3.4. Зміст звіту

3.4.1. Мета роботи.

3.4.2. Завдання до роботи.

3.4.3. Система типів.

3.4.4. Перелік семантичних перевірок.

3.4.5. Текст програми з основними коментарями.

3.4.6. Інтерфейс роботи з програмою в декількох режимах.

3.4.7. Результати тестування: вхідні дані та результати роботи програми.

3.4.8. Перелік проблем, які були виявлені і розв'язані або не розв'язані під час роботи над проектом, з описом відповідної ситуації. Якщо проблему було розв'язано, то необхідно додатково описати прийняті рішення. Якщо проблему не було розв'язано, ситуацію має бути ідентифіковано максимально детально з поясненням причин.

3.4.9. Висновки, що відображають особисто отримані результати виконання роботи та їх критичний аналіз.

3.5. Контрольні запитання

3.5.1. Які завдання виконуються на етапі семантичного аналізу?

3.5.2. Яким чином виконується перевірка типів?

3.5.3. Що таке таблиця символів та яка її структура?

3.5.4. Яким чином використовується таблиця символів під час семантичного аналізу?

3.5.5. Яким чином виконати пошук всіх семантичних помилок замість пошуку однієї помилки за один раз?

3.5.6. На яких аксіомах та правилах ґрунтується семантичний аналіз?

3.5.7. Які засоби генератора ANTLR призначені для виконання семантичного аналізу?

3.5.8. На яких етапах компіляції виконується розподіл пам'яті?

3.5.9. Які бувають перетворення типів?

3.5.10. Що таке еквівалентність типів? Наведіть приклади.

ЛАБОРАТОРНА РОБОТА № 4 ГЕНЕРАТОР КОДУ

4.1. Мета роботи

4.1.1 Ознайомитись з архітектурою та нотацією машинних команд, які використовуються в сучасних процесорах.

4.1.2 Вивчити основні способи та методи, які використовуються у процесі трансляції програмного коду вхідною високорівневою мовою програмування у машинний код.

4.1.3. Навчитися проектувати та розробляти генератор коду у складі компілятора.

4.2. Короткі теоретичні відомості

У результаті виконання етапу генерації коду відбувається формування коду цільовою мовою (у вигляді низькорівневих машинних команд).

Для виконання генерації коду спочатку створюється проміжне представлення коду, а потім вже формується низькорівневе представлення. Проміжне представлення, наприклад, легко зрозуміти на основі розбору складного виразу.

Якщо вираз представляє собою певне стандартне обчислення, наприклад, $x = 100 * 5 + 2 * 10$, то спочатку такий вираз має бути розбитий на простіші вирази, а потім має бути збережено остаточний результат. Оскільки кожен з проміжних виразів потребує подальшого використання його результату, то необхідно ввести додаткові змінні. Тоді перший рядок проміжного представлення можна визначити як $x1 = 100 * 5$. Другий рядок тоді представляється виразом $x2 = 2 * 10$. Відповідно третій вира формується як $x = x1 + x2$. Кожна з цих дій потребує у свою чергу розкладання на декілька базових операцій низькорівневою мовою, враховуючи наявні для відповідної архітектури команди. Так перший вираз має бути виконано через внесення значення (одного операнду) в регістр, потім використання знакового множення на другий операнд виразу. Після чого отриманий результат і є результатом обчислення першого виразу.

4.3. Завдання на лабораторну роботу

4.3.1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.

4.3.2. Вивчити документацію на процесор (з детальним переліком машинних команд для даної архітектури процесора), архітектура якого відповідає тому комп'ютеру, на якому планується використовувати даний компілятор.

4.3.3. Розробити програмне забезпечення, яке виконує генерацію машинного коду на основі дерев, якими представлений програмний код після виконання етапів аналізу тексту програми вхідною мовою, у два етапи: спочатку створюючи проміжне низькорівневе представлення коду, а потім перетворюючи його на машинні інструкції. Під час створення проміжного представлення інтегрувати декілька технік оптимізації коду.

4.3.4. Розширити функціональність розробленого програмного забезпечення можливістю перетворення згенерованого машинного коду у виконуваний файл. Для цього використовувати існуючі готові інструменти (зокрема gcc для Unix-подібних операційних систем).

4.3.5. Виконати тестування побудованого генератора коду.

4.3.6. Оформити звіт.

4.3.7. Відповісти на контрольні запитання.

4.4. Зміст звіту

4.4.1. Мета роботи.

4.4.2. Завдання до роботи.

4.4.3. Використана архітектура процесора та перелік інструкцій.

4.4.4. Опис способу перетворення високорівневого представлення програмного коду на машинні команди.

4.4.5. Текст генератора коду з основними коментарями.

4.4.6. Інтерфейс роботи з програмою в декількох режимах.

4.4.7. Результати тестування: вхідні дані та результати роботи програми.

4.4.8. Перелік проблем, які були виявлені і розв'язані або не розв'язані під час роботи над проектом, з описом відповідної ситуації.

Якщо проблему було розв'язано, то необхідно додатково описати прийняті рішення. Якщо проблему не було розв'язано, ситуацію має бути ідентифіковано максимально детально з поясненням причин.

4.4.9. Висновки, що відображають особисто отримані результати виконання роботи та їх критичний аналіз.

4.5. Контрольні запитання

4.5.1 Що таке трьохадресний код?

4.5.2 Які існують проміжні представлення коду? Наведіть приклади.

4.5.3 Чи відрізняється машинний код для одного й того самого процесора, якщо використовуються різні операційні системи?

4.5.4 Які завдання повинні бути виконані на етапі генерації машинного коду?

4.5.5 Яким чином перетворити умовні оператори та інструкції вибору на машинні інструкції?

4.5.6 Чи відрізняється розміщення в пам'яті глобальних та локальних змінних?

4.5.7 Яким чином виконується трансляція звернень до масиву?

4.5.8 Яким чином відбувається трансляція операторів циклу?

4.5.9 Що таке середовище часу виконання та яким чином воно реалізується?

4.5.10 Які дані зберігаються в стеку, а які в кучі?

ДОДАТОК А

ІНДИВІДУАЛЬНІ ЗАВДАННЯ

1. Розробити IDE для мови програмування Swift. Обов'язково мають бути проаналізовані наступні можливості даної мови: оголошення змінних, констант, масивів, асоціативних масивів, класів, цикли, умовні оператори, функції, робота з об'єктами, арифметичні оператори та оператори присвоювання, коментарі.

Література:

The Swift Programming Language (5.10 beta) [Електронний ресурс]. – Режим доступу : <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>

2. Розробити IDE для мови програмування Java. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, примітивні типи (byte, int, double), арифметичні оператори, оператори присвоювання, масиви, оголошення змінних, класів, наслідування, використання циклів (while) і умовних операторів (switch), оброблення об'єктів, коментарі.

Література:

The Java Tutorials [Електронний ресурс]. – Режим доступу : <https://docs.oracle.com/javase/tutorial/index.html>

3. Розробити IDE для мови програмування Python. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, вбудовані типи об'єктів (динамічна типізація на основі булевого типу та цілих чисел), словники, кортежі, виконання арифметичних операцій і операцій присвоювання, цикли (while), умовні інструкції, оголошення функцій, визначення класів, робота з об'єктами, форматування блоків коду за відступами.

Література:

Our Documentation. Python.org [Електронний ресурс]. – Режим доступу : <https://www.python.org/doc/>

4. Розробити IDE для мови програмування Scala. Обов'язково мають бути проаналізовані наступні можливості даної мови: оголошення змінних, арифметичні операції і операції присвоювання, масиви, переліки, створення об'єктів, високорівневі і анонімні функції, умовні оператори і цикли.

Література:

Learn Scala. Scala Documentation [Електронний ресурс]. – Режим доступу : <https://docs.scala-lang.org/>

Introduction. Scala 3 – Book. Scala Documentation [Електронний ресурс]. – Режим доступу : <https://docs.scala-lang.org/scala3/book/introduction.html>

5. Розробити IDE для мови програмування Rust. Обов'язково мають бути проаналізовані наступні можливості даної мови: вбудовані типи даних, функції, вектори, записи, методи, цикли, умовні оператори, зв'язування змінних, арифметичні оператори.

Література:

Klabnik, S. The Rust Programming Language [Електронний ресурс] / Steve Klabnik and Carol Nichols. – Режим доступу : <http://doc.rust-lang.org/book/>

Learn Rust – Rust Programming Language [Електронний ресурс]. – Режим доступу : <https://www.rust-lang.org/learn>

6. Розробити IDE для мови програмування C++. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, оголошення змінних, констант і масивів, виконання арифметичних операцій і операцій присвоювання, функції, елементарні математичні операції і операції присвоювання, використання циклів і умовних операторів, коментарі.

Література:

C++ Language [Електронний ресурс]. – Режим доступу : <https://cplusplus.com/doc/tutorial/>

7. Розробити IDE для мови програмування Ruby. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, вбудовані типи об'єктів (динамічна типізація), оголошення класів, виконання арифметичних операцій, операцій присвоювання, цикли, умовні інструкції, ітератори, робота з об'єктами, логічні вирази, коментарі.

Література:

Programming Ruby The Pragmatic Programmer's Guide [Електронний ресурс] / Режим доступу : <https://ruby-doc.com/docs/ProgrammingRuby/>

Documentation [Електронний ресурс] / Режим доступу : <https://www.ruby-lang.org/en/documentation/>

8. Розробити IDE для мови програмування Java. Обов'язково

мають бути проаналізовані наступні можливості даної мови: ідентифікатори, примітивні типи (void, boolean, char, float), арифметичні оператори, оператори присвоювання, логічні оператори, оголошення змінних, класів, використання циклів (for) і умовних операторів (if), оброблення об'єктів, коментарі.

Література:

The Java Tutorials [Електронний ресурс]. – Режим доступу : <https://docs.oracle.com/javase/tutorial/index.html>

9. Розробити IDE для мови програмування Cool (Classroom Object-Oriented Language). Мають бути реалізовані всі можливості даної мови. Програми мовою Cool є множинами класів:

```
class <type> [ inherits <type> ] {
    <feature_list>
};
```

Література:

Aiken, A. The Cool Reference Manual [Електронний ресурс] / A. Aiken. – Режим доступу : <https://theory.stanford.edu/~aiken/software/cool/cool-manual.pdf>

10. Розробити IDE для мови програмування C#. Обов'язково мають бути проаналізовані наступні можливості даної мови: оголошення класів, ідентифікатори, основні типи даних, масиви, арифметичні оператори, оператори присвоювання, відносні оператори, оголошення змінних, оператори вибору і циклів, оброблення об'єктів, коментарі.

Література:

C# docs – get started, tutorials, reference. Microsoft Learn [Електронний ресурс] / Режим доступу : <https://learn.microsoft.com/en-us/dotnet/csharp/>

11. Розробити IDE для мови програмування Go. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, вбудовані типи даних, оголошення змінних, масивів, функцій, структур, класів та відповідні операції над ними, цикли, умовні оператори.

Література:

Documentation – The Go Programming Language [Електронний ресурс] / Режим доступу : <https://golang.org/doc/>

12. Розробити IDE для мови програмування Pascal. Обов'язково мають бути проаналізовані наступні можливості даної мови: стандартна структура програми, ідентифікатори, прості типи даних, переліки, оголошення змінних, арифметичні оператори та оператори присвоювання, масиви, множини, використання циклів і умовних операторів, покажчики, визначення власних типів, функції та процедури, коментарі.

Література:

Canneyt, M. V. Reference guide for Free Pascal, version 3.2.2 [Електронний ресурс] / M. V. Canneyt. – 2021. – Режим доступу : <https://www.freepascal.org/docs-html/current/ref/ref.html>

13. Розробити IDE для мови програмування Visual Basic. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, оголошення змінних, основні типи даних, арифметичні оператори, оператори присвоювання, порівняння, керуючі конструкції, цикли, масиви, класи, функції та процедури, коментарі.

Література:

Visual Basic docs – get started, tutorials, reference. Microsoft Learn [Електронний ресурс] / Режим доступу : <https://learn.microsoft.com/en-us/dotnet/visual-basic/>

14. Розробити IDE для мови програмування Python. Обов'язково мають бути проаналізовані наступні можливості даної мови: ідентифікатори, вбудовані типи об'єктів (динамічна типізація на основі рядків та чисел з плаваючою крапкою), переліки, множини, виконання арифметичних операцій і операцій присвоювання, цикли (for), виключення, оголошення функцій і зокрема lambda-функцій, робота з об'єктами, форматування блоків коду за відступами, логічні вирази, коментарі.

Література:

Our Documentation. Python.org [Електронний ресурс]. – Режим доступу : <https://www.python.org/doc/>

ЛІТЕРАТУРА

1. Thain, D. Introduction to Compilers and Language Design [Текст]: Second Edition / D. Thain. – 2020. – 240 p.
2. Cooper, K. Engineering a Compiler [Текст] / Keith Cooper, Linda Torczon ; Third Edition. – Morgan Kaufmann, 2022. – 848 p.
3. Compilers [Текст]: Principles, Techniques, and Tools / Alfred Aho, Jeffrey Ullman, Ravi Sethi, Monica Lam. – Addison Wesley, 2006. – 1040 p.
4. Parr, T. The Definitive ANTLR 4 Reference [Текст]: Second Edition / T. Parr. – Pragmatic Bookshelf, 2013. – 328 p.
5. Jeffery, C.L. Build Your Own Programming Language [Текст]: A programmer's guide to designing compilers, interpreters, and DSLs for solving modern computing problems / C.L. Jeffery. – Packt Publishing, 2021. – 494 p.
6. Rosenberg, J.B. How Debuggers Work [Текст]: Algorithms, Data Structures, and Architecture / Jonathan B. Rosenberg. – John Wiley & Sons, 1996. – 272 p.
7. Nystrom, R. Crafting Interpreters [Текст] / R. Nystrom. – Genever Benning, 2021. – 639 p.
8. ANTLR [Електронний ресурс]. – Режим доступу: <https://wwwantlr.org/>
9. ANTLR 4 Documentation [Електронний ресурс]. – Режим доступу: <https://github.com/antlr/antlr4/blob/master/doc/index.md>
10. Thain, D. Introduction to Compilers and Language Design, 2nd edition [Електронний ресурс] / D. Thain. – 2020. – Режим доступу: <https://www3.nd.edu/~dthain/compilerbook/>
11. Appel, A.W. Modern Compiler Implementation in Java [Текст] / Andrew W. Appel, Jens Palsberg. – Second Edition. – Cambridge University Press, 2002. – 501 p.
12. Siek, J. G. Essentials of Compilation [Текст] : An Incremental Approach in Python / J. G. Siek. – Cambridge : The MIT Press, 2023. – 232 p.